

BhūDrishti Agentic Command — Master Build Prompt for Antigravity IDE

SIH PS20: Agentic Disaster Relief & Emergency Resource Coordinator

CONTEXT (read fully before writing any code)

We are upgrading an EXISTING working prototype called BhūDrishti — a satellite-intelligence chat tool — into a full multi-agent disaster resource-coordination system that satisfies SIH PS20. We are NOT starting from scratch and we are NOT removing anything that exists. The satellite layer becomes the evidence layer that grounds and verifies the new coordination layer. This dual-evidence design (satellite + human reports) is our core differentiator versus teams building coordination systems on self-reported data alone.

What already exists — do not rewrite the internals of these

Backend (FastAPI, Python):

- backend/main.py — single stateless endpoint POST /api/chat. Pipeline: detect_intent() (LLM-based module + location extraction, supports English/Hindi/Hinglish/misspellings) → parallel STAC search (stac_client.py: search_sentinel1_sar / search_sentinel2) + weather (get_real_weather) → parallel GEE compute (gee_client.py: calculate_real_ndvi, calculate_water_area, get_gee_map_tile) → verify_flood_output() — an existing NDMA historical-baseline sanity-check agent (dict HISTORICAL_FLOOD_RECORDS keyed by state, returns confidence HIGH/MEDIUM/LOW + reasoning notes) → Gemini Vision analysis (vlm_client.py: analyze_image_with_gemini / analyze_image_with_nvidia) → optional translate_text() via NVIDIA Riva. Also has GET /api/health and GET /api/basemap.
- No database. No persistence. No concept of zones/reports/inventory/agencies/allocations anywhere today — every request is stateless.

Frontend (Next.js + React + TypeScript):

- frontend/src/app/layout.tsx — loads CesiumJS via a raw <script> tag (not an npm dependency — check here, not package.json, if verifying Cesium is available).
- frontend/src/components/MapPanel.tsx — working Cesium 3D globe: ArcGIS World Imagery base layer, ArcGIS Terrain3D elevation, ESA/GEE tile overlays, a working draw-region tool (toggleDraw, drawEntityRef, emits GeoJSON via onDrawComplete prop), and a RainViewer weather-radar overlay toggle (showWeather).

- frontend/src/components/ChatPanel.tsx — chat UI with a working PDF export function (downloadPDF) that generates a "Classified PDF Intelligence Report" from a chat response.
- frontend/src/app/page.tsx — layout with a module nav (flood/agri/urban/forest/water), data-source list, AI-model list.
- frontend/src/lib/api.ts — ChatRequest/ChatResponse types, calls API_BASE + /api/chat.

SECURITY FIX — do this before Phase 1

backend/main.py and backend/vlm_client.py currently have a hardcoded Gemini API key as the fallback default in os.environ.get("GEMINI_API_KEY", "AQ.Ab8RN6..."). This repo is public on GitHub, so that key is exposed right now.

- Remove the fallback string entirely — require GEMINI_API_KEY from .env only, and raise a clear startup error if it's missing.
- Confirm .env is in .gitignore.
- Note in your PR/commit message that the key should be rotated in Google AI Studio (this is a manual step for the team, not something you can do from code).

THE PS20 REQUIREMENT MAP

PS20 requirement	Where it's built below
Incident/zone reporting interface	Phase 8: ZoneReportPanel.tsx
Resource inventory management	Phase 1 (store) + Phase 7 (/api/resources, adjust endpoint)
Needs-assessment agent	Phase 3: needs_agent.py
Allocation/optimization agent	Phase 4: allocation_agent.py
Inter-agency coordination workflow	Phase 5: agency_coordinator.py + accept/reject/status endpoint
Dynamic re-allocation on new reports	Phase 6: reallocation_agent.py
Priority/severity scoring	Phase 2: severity_scorer.py

PS20 requirement	Where it's built below
Duplicate-effort detection	Phase 4 refinement: 3W/4W matrix (see Research Refinements)
Coordination dashboard	Phase 8: CommandDashboard.tsx
Activity/audit log	Phase 1 (audit_log list) + Phase 8 (UI panel)

BUILD IN PHASES. Show output after each phase before proceeding.

PHASE 1 — State store & data models

New file: backend/store/state.py

- Simple in-memory Python dict state, persisted to state.json on every mutation (load-on-startup, save-on-write — no ORM, no DB server; this is a deliberate hackathon-scope tradeoff, confirm you agree before building, flag if not).
- zones: dict[zone_id -> Zone]
- inventory: dict — resource stocks by type (food_kg, water_liters, medical_kits, shelter_units, ndr_f_teams, army_personnel, ngo_units), each tracked as {available, reserved} not a flat number.
- assignments: list[Assignment]
- audit_log: list — append-only: {timestamp, agent, event_type, zone_id, description}
- coordination_matrix: list — see Phase 4 refinement (3W/4W table)

New file: backend/models/zone.py

```
class ZoneReport(BaseModel):
```

```
    zone_id: str
```

```
    location: str
```

```
    severity_reported: int    # 1-10, human input
```

```
    population: int
```

```
    zone_area_km2: Optional[float] # for exposure weighting, Phase 2
```

```
    needs: dict                # {"food": 1000, "ndrf": 3, ...}
```

```
    description: str
```

```
geojson: Optional[dict]
gee_area_km2: Optional[float] # injected from GEE
ndvi_score: Optional[float] # injected from GEE
rainfall_mm: Optional[float] # injected from weather API
severity_final: Optional[float] # computed by severity_scorer
satellite_agreement_confidence: Optional[float] # 0-1, Phase 2
cycles_unfulfilled: int = 0 # equity guard, Phase 4
status: str # "active" | "responding" | "resolved"
source: str = "official" # "official" | "crowd" (Phase 9 stretch)
```

New file: backend/models/resource.py — ResourceInventory, ResourceType enum.

New file: backend/models/agency.py

```
class Assignment(BaseModel):
```

```
    id: str
    zone_id: str
    agency_id: str
    resource_type: str
    quantity: float
    status: str # proposed | accepted | rejected | in_transit | delivered
    reasoning: str
    time_window_start: datetime
    time_window_end: datetime
    conflict_flagged: bool = False
    created_at: datetime
    superseded_by: Optional[str] = None
```

All Pydantic models get matching schemas in backend/schemas.py for request/response validation (needed for Phase-6 structured-output enforcement below).

PHASE 2 — Severity Scorer (research-grounded formula)

backend/agents/severity_scorer.py — weighted formula:

- **30%** reported human severity (severity_reported)
- **40%** GEE SAR flood area, but EXPOSURE-WEIGHTED, not raw area:
exposure_factor = gee_area_km2 * (population / zone_area_km2) when zone_area_km2 is available, else fall back to raw gee_area_km2 normalized against the existing HISTORICAL_FLOOD_RECORDS baseline already in main.py (import it, do not duplicate the dict). Rationale: a flood over farmland and an identical-area flood over a dense town are not equally severe — raw area alone is a known simplification in flood-damage literature; this is a lightweight fix appropriate for hackathon scope. Note in code comments that building/land-use-level damage estimation is the natural next step, out of scope here.
- **20%** population density
- **10%** live rainfall (reuse get_real_weather, already wired)

Also compute satellite_agreement_confidence (0.0–1.0): how closely the GEE-derived severity component and severity_reported align. **This must be a continuous score, not a boolean flag** — store it on the Zone record. Derive a simple HIGH/MEDIUM/LOW label on top for the dashboard badge, reusing the same three-tier pattern already used by verify_flood_output() in main.py for consistency — don't invent a second confidence vocabulary.

Return: severity_final (0–10) + a human-readable reasoning string (e.g. "Zone C scored 8.5/10: high population density, 0 medical teams available, satellite shows 340km² flooded vs 800km² typical for monsoon Assam"). Write to audit_log.

PHASE 3 — Needs-Assessment Agent

backend/agents/needs_agent.py

- Gemini call (reuse the client pattern already in vlm_client.py), **using function-calling / structured JSON schema output matching the Pydantic models from Phase 1** — not free-text parsing (see Research Refinement 6, non-negotiable, do this from the start rather than retrofitting later).
- Input: zone description + gee_area_km2 + ndvi_score.
- Output: structured needs dict.
- On Gemini failure/timeout or schema-validation failure: fall back to a formula-based estimate (population × per-capita constants) rather than passing malformed data downstream. Must not block the pipeline.

PHASE 4 — Allocation Agent + 3W/4W Duplicate Detection + Equity Guard

backend/agents/allocation_agent.py

Core allocation:

- Sort zones by severity_final DESC.
- Greedy match: for each zone, deduct from inventory.available, create Assignment (status="proposed") with a time_window_start/ time_window_end covering the expected response window.
- If inventory insufficient: set partial_allocation=true on the zone.

Duplicate-effort detection — 3W/4W matrix, not bbox overlap: This models UN OCHA's actual field-tested coordination tool (used since 2005, when the pre-cluster-system humanitarian response suffered from duplicated efforts and simultaneous coverage gaps — a real, citable precedent for judges). Track every active Assignment as a row in state.coordination_matrix: {agency, resource_type, zone_id, time_window_start, time_window_end}. Before finalizing a new Assignment, query this table for rows matching the same zone_id + resource_type with an overlapping time window from a DIFFERENT agency. If found, set conflict_flagged=true on both Assignments and log to audit_log with event_type="duplicate_flagged". Expose GET /api/coordination-matrix returning the full table (used by the dashboard in Phase 8).

Equity/starvation guard: Pure severity-greedy allocation will systematically starve persistently lower-severity zones — a documented failure mode in humanitarian resource-allocation research. Increment Zone.cycles_unfulfilled each allocation run where a zone's need isn't met. When a zone crosses a threshold (e.g. 3 unfulfilled cycles), apply a priority boost in the next severity_scorer run regardless of raw severity_final. Log this to audit_log as event_type="equity_boost" with before/after severity values — this is also a good demo talking point ("no zone gets permanently deprioritized").

Both the duplicate detector and equity guard must be re-runnable/ idempotent: re-running on unchanged state produces the same result; new data produces a diff (old Assignment's superseded_by points to the new one).

PHASE 5 — Agency Coordinator (explicit accept/reject workflow)

backend/agents/agency_coordinator.py

- Maps resource types → agency categories (NDRF / Army / NGO / Medical) and generates task orders with an ETA estimate from the nearest hardcoded depot (Guwahati / Delhi / Kolkata for demo).
- PATCH /api/assignments/{id}/status — accepts accepted | rejected | in_transit | delivered. **This endpoint is what actually satisfies PS20's "inter-agency coordination workflow" requirement** — a task order alone is a suggestion, not coordination, until an agency can act on it.
- On status=rejected: automatically trigger Phase 6 re-allocation for that zone/resource.
- On every status change: write to audit_log.

PHASE 6 — Re-Allocation Agent (with anti-thrashing cooldown)

backend/agents/reallocation_agent.py — triggers on:

- New zone report with higher severity than an existing allocated zone
- Updated satellite pass showing a larger-than-expected GEE area for an existing zone
- An Assignment rejected (Phase 5)
- **Inventory manually adjusted** — PS20 explicitly names changing "conditions," not just new reports, as a trigger. Add POST /api/resources/{type}/adjust and wire it here.

Anti-thrashing cooldown: before pulling resources from a zone that was allocated to within the last N minutes (configurable, e.g. 10 sim-minutes for demo), require the triggering zone's severity to exceed the target zone's by a minimum margin (e.g. >2.0 points) — not just be marginally higher. This prevents resources ping-ponging between two zones as scores fluctuate slightly, a known failure mode in dynamic allocation systems. Log any blocked attempt to audit_log as event_type="reallocation_blocked_cooldown" — this itself is a good demo moment ("the system prevents resource oscillation").

Steps: re-run severity_scorer on affected zones → if priority order changes beyond the cooldown margin, re-run allocation_agent → pull from lowest-priority active zones if needed → log before/after state with specific reasoning, e.g.: "Zone F severity (10.0) > Zone C severity (5.1) — reassigning 2 NDRF teams."

PHASE 7 — API endpoints (main.py — additive only)

POST /api/zones/report → submit report, runs Phase 2-4 agents

GET /api/zones → all zones + scores + confidence + conflict flags

GET /api/resources → current inventory (available/reserved)

POST /api/resources/{type}/adjust → manual inventory change (Phase 6 trigger)

GET /api/assignments → all task orders

PATCH /api/assignments/{id}/status → agency accept/reject/update (Phase 5)

GET /api/coordination-matrix → full 3W/4W table (Phase 4)

GET /api/audit-log → full decision trail, filterable by zone

POST /api/simulate → inject the 5 demo zones (Phase 9)

POST /api/simulate/inject-urgent → inject Zone F for live reallocation demo

POST /api/zones/{id}/resolve → mark resolved, release reserved resources

POST /api/reports/social → [STRETCH, Phase 9b] crowd-sourced report intake

Do not touch /api/chat, /api/health, /api/basemap — they stay exactly as-is and remain reachable from the existing Satellite Intel page.

PHASE 8 — Frontend: mode switcher + Command Dashboard

page.tsx: add a mode switcher in the navbar: [ Satellite Intel] [ Command Center]

- Satellite Intel mode = existing ChatPanel + MapPanel layout, completely unchanged.
- Command Center mode = CommandDashboard.tsx replaces ChatPanel; MapPanel gets a new commandMode: boolean prop.

CommandDashboard.tsx [NEW] — 3-column layout:

- **Left (280px):** zone cards, severity-color-coded. Shows severity score, GEE-verified area, population, allocated resources, and a confidence badge derived from satellite_agreement_confidence ( Satellite Verified /  Conflict Flagged — reuse the HIGH/MEDIUM/ LOW pattern from Phase 2, don't invent new labels).
- **Center:** existing Cesium globe with commandMode=true — pulsing zone markers (severity-colored), flow lines (billboard entities) from depots to zones, agency pins. Reuse existing draw/weather/layer toggles — they must keep working in this mode too.

- **Right:** resource inventory bars, agent pipeline status (NeedsAgent → AllocationAgent → Coordinator → ReAllocationAgent), scrollable audit log, and a small coordination-matrix table (Phase 4 data) so judges can visually see duplicate flags getting caught.

MapPanel.tsx: add commandMode prop — when true, render zone markers + agency pins from props alongside existing functionality.

ZoneReportPanel.tsx [NEW]: floating form — zone name/location, severity slider (color feedback), resource checkboxes, population, description, "Draw Zone on Map" button (reuse existing toggleDraw), submit → POST /api/zones/report. After submit: animated "Agents Running..." pipeline while polling for the result.

AgentStatusBar.tsx [NEW]: thin bottom bar, visible in Command mode, showing each agent's state (IDLE / RUNNING / DONE) with simple CSS animations (spinner for RUNNING, checkmark for DONE).

Polling: frontend polls GET /api/zones and GET /api/audit-log every 5s in Command Center mode. No WebSockets — simpler and more demo-reliable; confirm you agree before building, flag if not.

PHASE 9 — Demo simulation

POST /api/simulate seeds exactly these 5 zones with **pre-computed** GEE values (no live GEE call — must return instantly and reliably for a live judging session):

Zone Location		Severity Key Need		SAR Area
A	Jorhat, Assam	9.2	NDRF (5) + Medical	2,847 km ²
B	Silchar, Assam	7.8	Food (80,000 kg) + Shelter (500)	1,230 km ²
C	Guwahati, Assam	5.1	Water (200,000 L) + Medical kits	340 km ²
D	Dibrugarh, Assam	8.4	NDRF (4) + Army (200)	1,890 km ²
E	Dhubri, Assam	6.3	Shelter (300) + NGO	780 km ²

POST /api/simulate/inject-urgent injects Zone F (Barpeta, Assam, severity 10.0, 3,200 km²) to demonstrate live re-allocation. Expected behavior: reallocation_agent pulls 2 NDRF teams from Zone C (lowest severity, outside cooldown window) and logs "Zone F severity (10.0) > Zone C severity (5.1) — reassigning 2 NDRF teams" to audit_log, with Zone C marked partial_allocation=true and its cycles_unfulfilled counter incremented (setting up the equity-guard demo on a second injected event, if time allows showing that too).

Live ChatPanel queries still hit real GEE/STAC as before — only the simulate endpoints use pre-baked data, purely for demo reliability.

PHASE 9b — [STRETCH, only if Phases 1–9 are solid] Crowd-sourced report channel

POST /api/reports/social — accepts free-text "public reports" (pre-seeded sample messages for demo, no live scraping needed). Runs through the same needs_agent pipeline but tags source="crowd" and gives these a lower default weight in the severity blend unless corroborated by an official report or satellite evidence. This demonstrates three-way evidence fusion (official + satellite + crowd), which crisis-informatics research treats as standard — a good differentiator if judges probe report reliability, but genuinely optional if you're short on time.

NON-FUNCTIONAL REQUIREMENTS (apply throughout, not just at the end)

- Every agent decision writes to audit_log with clear reasoning — this is explicitly a judged PS20 criterion (audit/activity log).
- Enforce structured/schema-validated LLM outputs at every agent boundary from Phase 3 onward (not retrofitted later) — this follows a documented finding that multi-agent systems with enforced schemas between agents produce far more reliable output than free-text handoffs.
- Do not break /api/chat, /api/health, /api/basemap, or the existing Satellite Intel page — they remain fully functional and reachable one mode-switch away from Command Center.
- Confirm before building, flag if you disagree with any of these hackathon-scope tradeoffs: (1) JSON-file state store instead of a real DB; (2) 5s polling instead of WebSockets; (3) hardcoded GEE values only for /api/simulate, live GEE stays wired for real chat queries.

DEMO NARRATIVE — two lines worth saying explicitly to judges

1. *"Our duplicate-effort detection follows UN OCHA's actual 3W/4W coordination-matrix standard, used in humanitarian response since 2005 — not an invented geo-overlap heuristic."*
 2. *"Every agent boundary in our pipeline enforces structured schema validation rather than free-text handoffs, because that's what the recent multi-agent-orchestration literature shows is the difference between reliable and unreliable agentic systems in production incident-response settings."*
-

BUILD ORDER SUMMARY

Security fix → Phase 1 (state/models) → Phase 3's structured-output pattern established early → Phase 2 (severity) → Phase 3 (needs) → Phase 4 (allocation + 3W/4W + equity) → Phase 5 (coordinator + accept/reject) → Phase 6 (reallocation + cooldown) → Phase 7 (endpoints) → Phase 8 (frontend dashboard) → Phase 9 (demo seed) → Phase 9b if time remains.

Start with the security fix, then Phase 1. Show me state.py, models/zone.py, models/resource.py, and models/agency.py before proceeding to Phase 2.